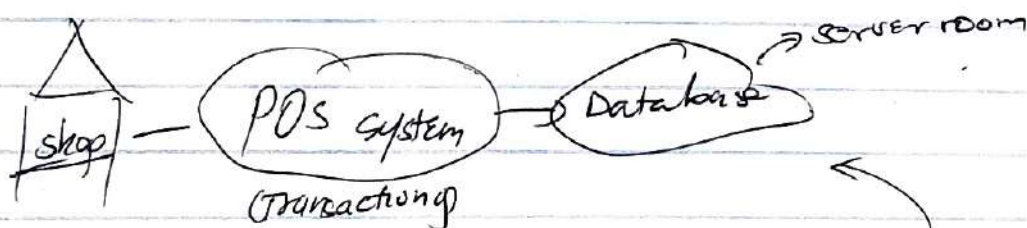
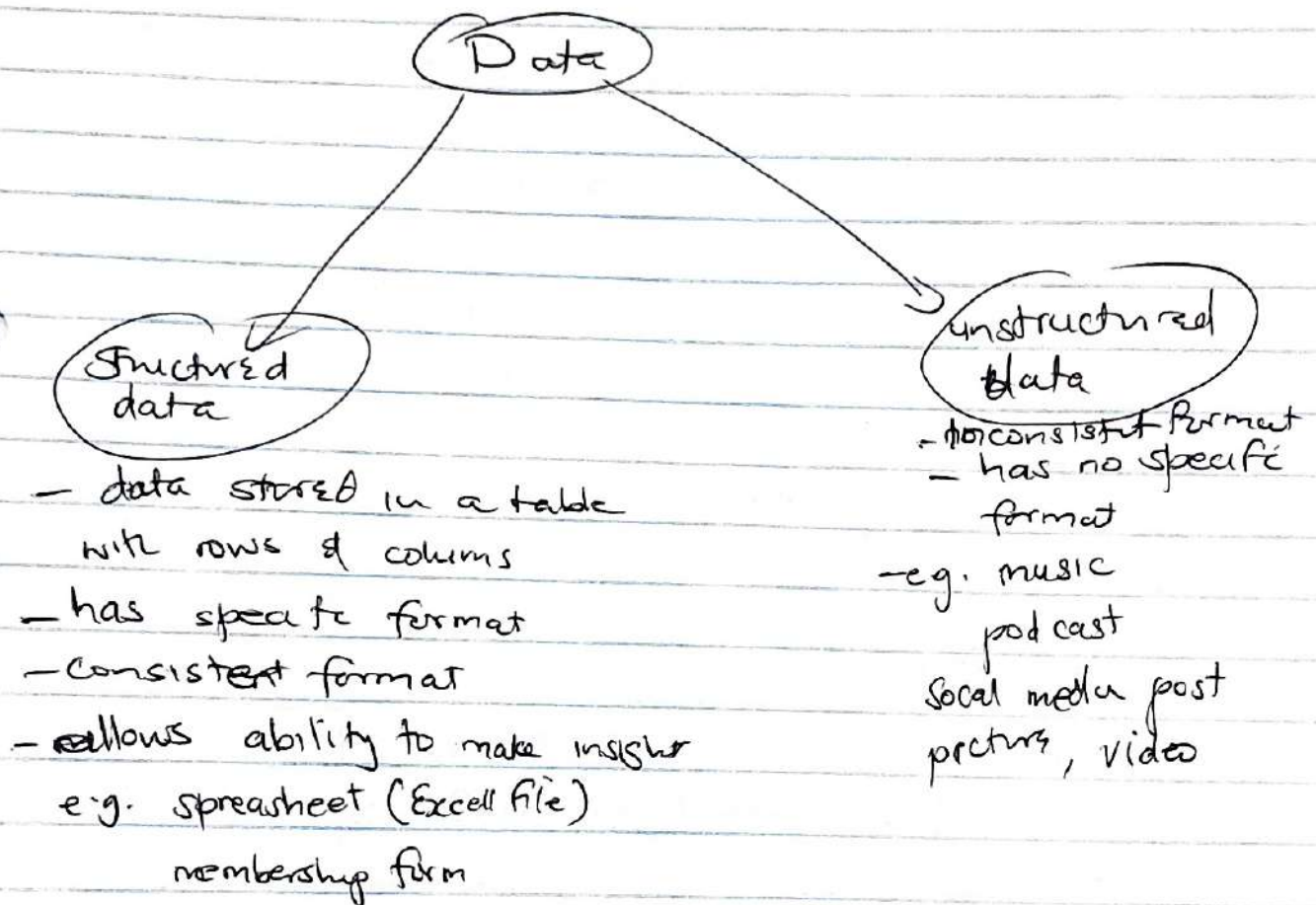


03 June 2026 Class

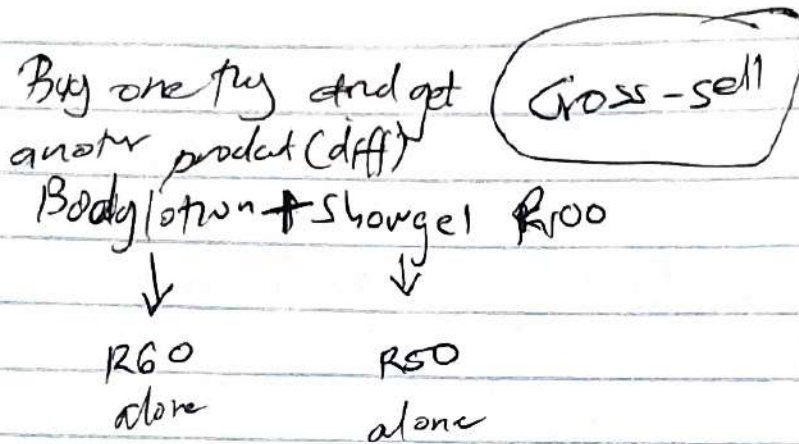
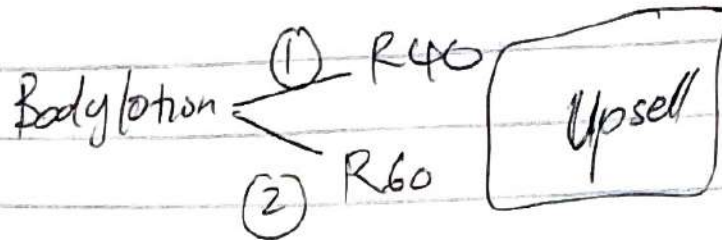
Intro to databases

- Data → collection of facts (information)



* Items purchased by you, record u keep track why you see adverts on socials of someth you just bought or was searching

* Whenever they data, forces time



Primary VS foreign key

* Example

- Student table
 - primary key : Student ID
 - Columns : Student ID, Name, Age

Student

- Course Table
 - foreign key : ^{student ID} refers to student ID in the student table
 - columns :

04 JUNE 2026 CLASS

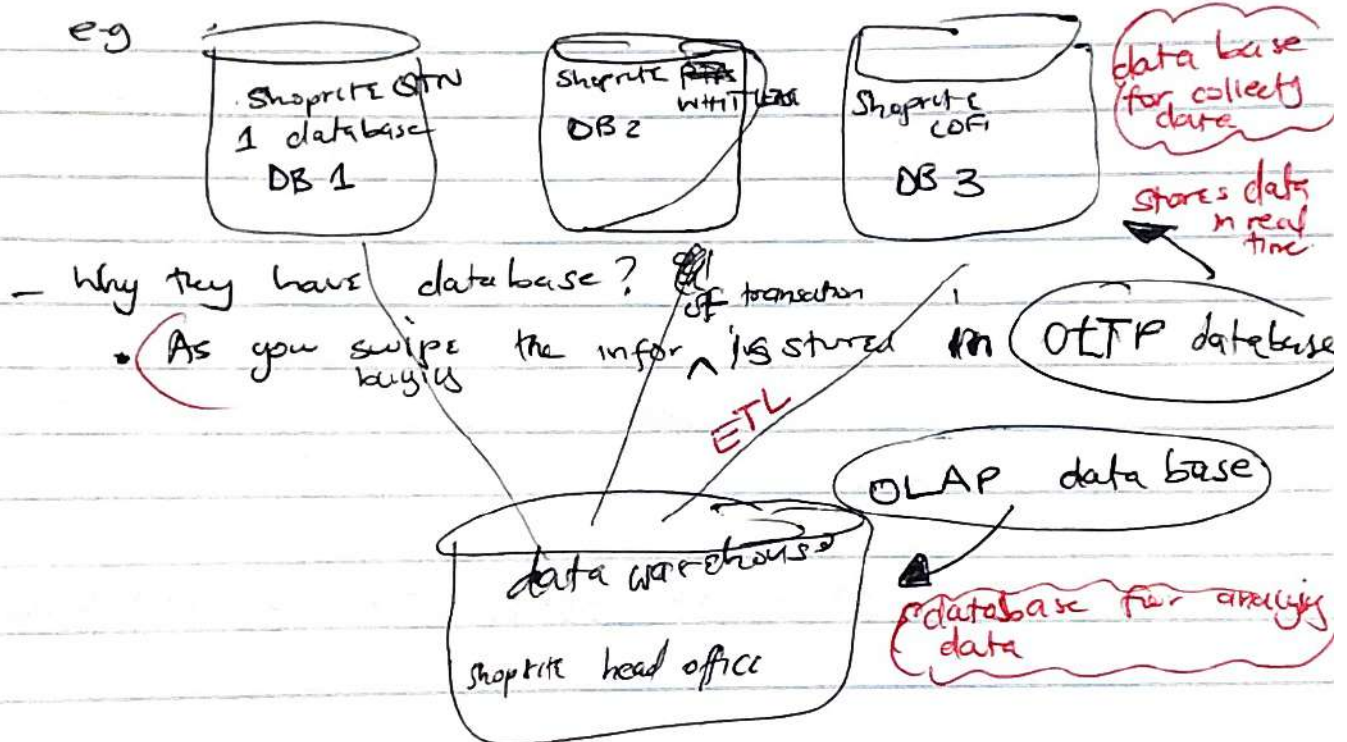
Structured data

- * As data is collected, each data point is distinguished by ID

Data flow

Data flows from sources to warehouse

eg



Data moves through ETL to data warehouse

* OLTP → is not used for analysis data but to capture the data

→ used to capture transactional data

→ used basic SQL

• create, insert, update, delete

* OLAP → used by data analyst & scientists

→ allows analysis of captured data

04 JUNE 2026 class

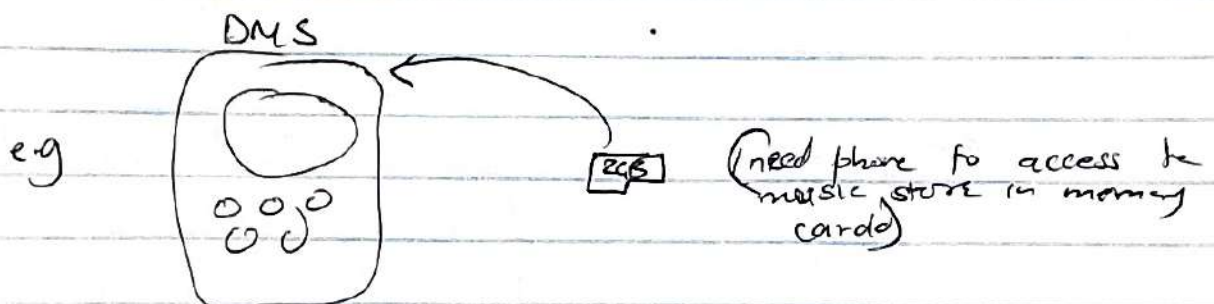
ETL → ensures data is available in Datawarehouse for analysis

→ system built to transport data from one point to another (from source to warehouse)

(this transfer of data is done by data engineer)

Data management system (DMS)

⇒ allows interaction with what's stored in the data warehouse.



⇒ system that gives us access to our data

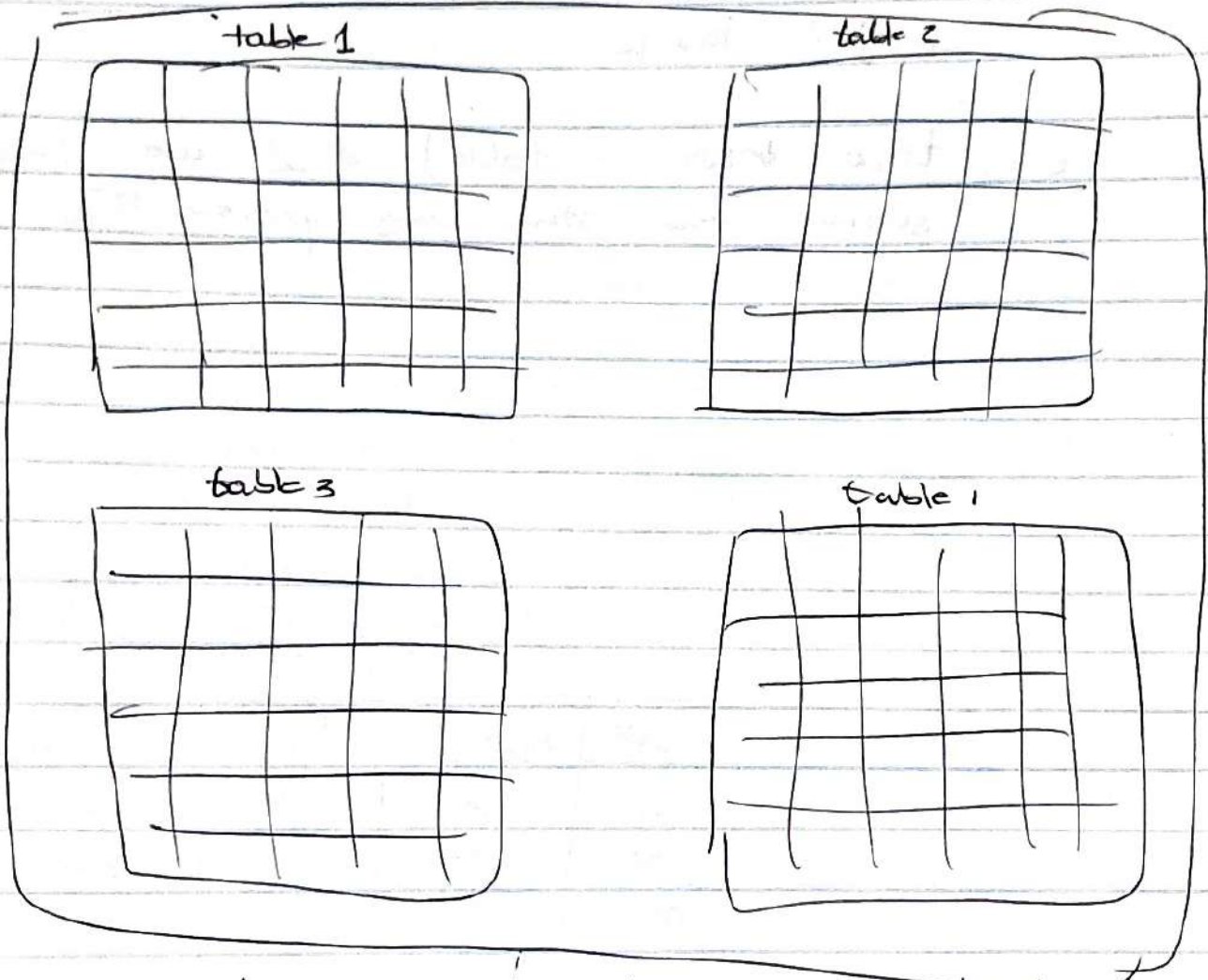
05/06/2026

7-9pm

Data Visualization Bootcamp

Data story telling

Datawarehouse



⇒ These table have a relationship with one another.

Relational database

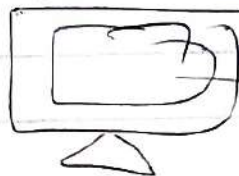
→ structured data that can be related
(or joined together) ~~to become~~ ~~useful~~

Why add → to see what data is available

→ Data exploring analysis

to understand the
relationship between
data points.

Data management system
(DMS)



→ e.g. databricks
snowflake
microsoft fabric

→ in a DMS we code
so we can clean our
data

→ we use DMS to access
data from warehouse

Customers

<u>Cust_id</u> ^{PK}	name	Age	city
1	Liso	29	NY
2	Xoxo	22	LA
3	Lita	18	MG

Dimensional table

Products

<u>Product_id</u> ^{PK}	Product-category	Product-name
101	fruits	Banana
102	fruits	Apple
103	Electronics	laptop
104	clothing	Uzzi jeans

⇒ these are dimension table (Dim)

- ↳ contains descriptive info
- ↳ does not ~~constantly~~ ^{consistently} change

PK → unique identifier that differentiate each ^{row} component from the table

→ (e.g. in SA it is your ID no.)

transactions table
fact table

order_id	Cust_id	Product_id	Rev
324	2	104	70
333	1	102	80
401	3	101	100

⇒ fact table → has foreign key
→ contains numbers
→ used for analysis

FK → is a primary key from one table that is found in another table

→ links to two tables

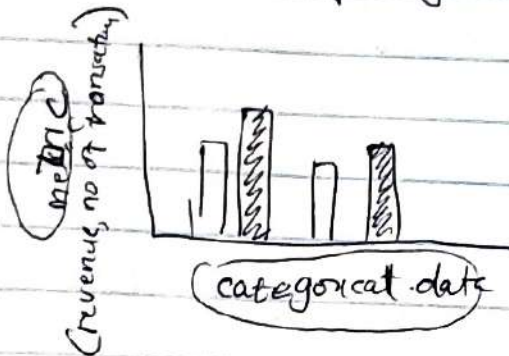
Graphs to use to build dashboards

The Data Design Blueprint

- choosing the right chart for effective story telling

① Bar graph/chart

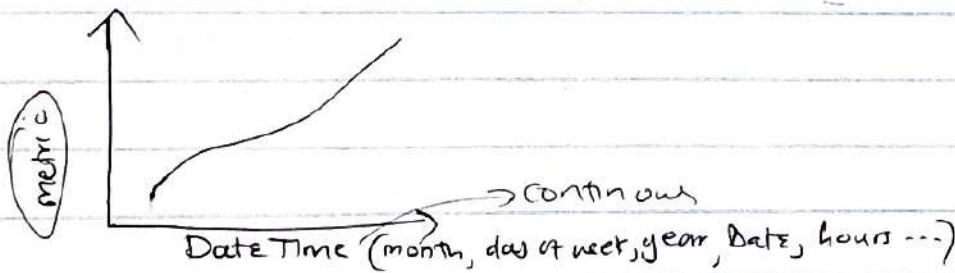
⇒ When comparing categories



* metric → something you are measuring
→ numerical data / quantitative data

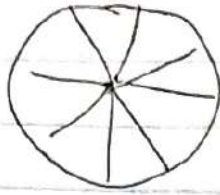
* Categorical data → grouping data (mainly words)

② Line chart



⇒ To show ^{continuous} trends over time

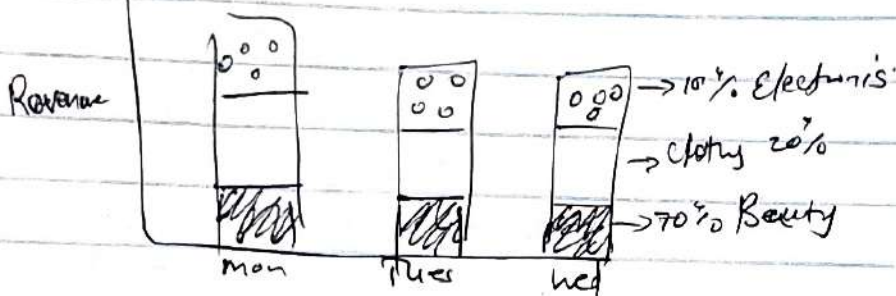
③ Showing proportions



⇒ used for categorical breakdown.

⇒ metric to proportion.

④ Stacked chart



⇒ To show proportions for 2 diff categorical data by a specific metric

Product category

- Electronics
- Beauty
- Clothing

Days of week category

- Mon
- Tues
- Wed

Excel Visualizations

- rows ⇒ x-axis

08/06/26

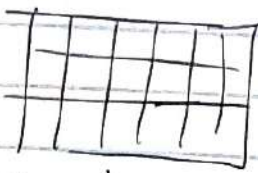
Introduction to SQL

Data + info we use to make business decisions

structured

unstructured

~~inconsistent~~



- stored in a table inside a data warehouse / database

When we have data we need to ~~extract~~ insights from it

Structured Query Language (SQL)

- ↳ is a scripting language used to interact with structured data
- ↳ used to retrieve data (information) from a structured table

SQL FUNDAMENTALS

08/06/28

~~Str~~ students_results

Student_id	name	marks
101	Liso	99
102	Xoxo	25
103	Sino	50
104	Mattembi	60

SELECT statement

allow us to select specific columns from our table
columns you want in your final output

FROM statement

allows you to specify the name of table we are selecting
the data from

```
SELECT name, marks  
marks  
  
FROM student_results;
```



name	marks
Liso	99
Xoxo	25
Sino	50
Mattem	60

SELECT *

SELECT STAR

is a way to select all the columns from
your table without specifying your columns one
by one

- Each column name is separated by a comma when selecting
specific columns from a table.

e.g

```
SELECT Date,  
Gender,  
Age  
  
FROM retail_sales_dataset;
```

NB: you don't put the comma after
~~the last column~~ specifying the last
column on SELECT

SQL FUNDAMENTALS ^{different/unique}

08/06/23

SELECT DISTINCT statement

- ↳ we want to get diff values from a specific column
- ↳ we remove duplicates / we don't repeat the same value / ^{clears} duplicates

Syntax : `SELECT DISTINCT column_name
FROM table_name;`

e.g.

product_id	products	Price
101	Apple	100
102	Orange	50
103	Apple	60
104	Banana	150

e.g. fruits_orders table

order_id	prd_name	Revenue
101	Apple	100
102	Orange	50
103	Apple	60
104	Banana	150

Q → Write a SQL code to give me the diff products this man is selling

Syntax : `SELECT DISTINCT prd_name
FROM fruit_orders;`



prd_name
Apple
Orange
Banana

AB apple shown once even though it appears twice on the table.

Fundamentals SQL continued

09/06/2026

• 3 ways of selecting data

- ① `SELECT *` → select all columns from our table
- ② `SELECT DISTINCT` → select diff values from table (not duplicates)

③ SELECTING SPECIFIC COLUMNS

`SELECT column 1,
column 2,
column 3
FROM table_name;`

→ we want to select specific columns from our table, we specify the columns we want.

SQL filtering statement (Data filtering)

- ↳ we filter data if we want to get specific information from our data. (e.g. only want data for 2026 not previous years....)
- ↳ is a way to exclude rows of data that do not meet a specific criteria.

* How to filter data?

- use WHERE statement

WHERE statement

Syntax:

~~SELECT~~
`SELECT column 1,
column 2,
column 3
FROM table_name
WHERE column condition;`

SQL FUNDAMENTALS

09/06/26

WHERE statment

→ similar to Vlookups in Excel

e.g students_results

Student_id	name	marks
101	Liso	99
102	Xoxo	25
103	Sino	50
104	Mafumbi	60

Q Write a SQL query to give me only the names of students that obtained more than 60% in the exam

```
SELECT * OR SELECT marks  
FROM students_results  
WHERE marks > 60;
```

→

student_id	name	marks
101	Liso	99

↳ Filtered table based on the condition

⇒ When you filter, the number of rows on the table become reduced.

- SQL operators (will be used for data filtering)

① Arithmetic operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	division

② Comparison operators

Operator	Description
=	Equal to
>	Greater than
<	less than
>=	Greater or equal to
<=	less or equal to
<>	Not equal to

③ Logical Operators

Operator	Description
AND	Both conditions must be met
OR	One condition must be met
BETWEEN	Must be within the range
IN	Must be contained in the list
NOT	Gives the opposite result of what is specified
LIKE	Looks for a pattern

Back to Data Filter

10/06/28

- Data filtering → **WHERE** statement

↳ is a filtering statement that we use to filter our data based on specific conditions.

- We can use **SQL operators** to build the condition for filtering our data.

(Within the WHERE statement we put the conditions we want)

• Syntax :

```
SELECT column_1,  
        column_2,  
        column_3  
FROM table_name  
WHERE condition;
```

• e.g

users

user_id	user_name	City
1	Liso	Johannesburg
2	Xoxo	Cape Town
3	Sino	Durban
4	Mothembu	Pretoria
5	Vuvu	Port Elizabeth

↳ value within column

Q: Write an SQL query to give me the name of users in Durban

A:

```
SELECT *  
FROM users  
WHERE city = 'Durban';
```

↳ Condition (we used an operator (=) to build the condition)

user_id	user_name	City
3	Sino	Durban

Continued

10/06/20

e.g

membership_plan

plan_id	plan_name	monthly_price
101	Basic	79
102	Standard	129
103	Premium	199
104	Family plan	249
105	Mobile	59

Q: Write an SQL query to give me the plans that cost more than 150

A:

```
SELECT plan_name,
       monthly_price
FROM   membership_plan
WHERE  monthly_price > 150;
```

result

Plan_name	monthly_price
Premium	199
Family	249

SQL fundamentals

15/06/28

• SQL Logical operators

- AND statement

- ↳ allow us to combine multiple conditions
- ↳ the conditions must be satisfied
- ↳ commonly used when building logical conditions for filtering data

Syntax: `SELECT column 1,
 column 2
FROM table_name
WHERE condition AND condition AND condition`

↳ e.g

students

student_id	name	Course	Age	City
1	Liso	Data analytics	22	Johannesburg
2	Mathemba	Data Science	19	Pretoria
3	Xoxo	Data analytics	25	Durban
4	Sino	Data analytics	20	Johannesburg
5	Vuvu	Data science	23	Johannesburg

Q Write an SQL query to give the names of students living in Johannesburg that are doing data analytics.

A

```
SELECT * name,  
            City } *  
FROM students  
WHERE city = 'Johannesburg'      ← condition 1  
      AND course = 'Data analytics';      ← condition 2
```

Student_id	name	Course	Age	City
1	Liso	Data analytics	22	Johannesburg
4	Sino	Data analytics	20	Johannesburg

- OR statement

- ↳ allows us to ~~multiple~~ combine multiple conditions
- ↳ one or the conditions must be met.

Q write an SQL query to give me names of students living in Johannesburg or students doing data analytics

A

```

SELECT *
FROM students
WHERE city = 'Johannesburg' OR
course course = 'Data analytics';
  
```

↓

Student_id	name	Course	Age	City
1	Liso	Data analytics ✓	22	Johannesburg ✓
3	Xoxo	Data analytics ✓	25	Durban X
4	Sino	Data analytics ✓	20	Johannesburg ✓
5	Vuvu	Data Science X	23	Johannesburg X

- IN statement

↳ It is a way of filtering data by specifying the values of interest from a specific column

↳ syntax:

```
SELECT *  
FROM table_name  
WHERE column IN ('Value1', 'Value2', 'Value3', ...)
```

no limit in the number of values you can specify

e.g. use to previous table

Q. write an SQL query to give me people living in Durban and Pretoria

A

```
SELECT name,  
        city  
FROM students  
WHERE city = 'Durban'  
WHERE city IN ('Durban', 'Pretoria');
```



name	city
Mattembi	Pretoria
Xoxo	Durban

- NOT IN statement



```
SELECT name,  
       city  
FROM student  
WHERE city NOT IN ('Durban', 'Pretoria');
```

NOT is an exclusion statement



Name	City
Liso	Johannesburg
Sino	Johannesburg
Vuvu	Johannesburg

BETWEEN statement

Between AND so
based on bound

↳ It allows us to filter the data based on a specific range of values

↳ It is an inclusive statement, this means that the lower and upper bounds of the between statement are included.

↳ syntax :

```
SELECT column 1,  
        column 2,  
        column 3  
FROM table_name  
WHERE column BETWEEN lower bound AND upper bound;
```

Q. write an SQL query to give me the name & age of students between the ages of 20 to 23

A.

```
SELECT name,  
        age  
FROM students  
WHERE age BETWEEN 20 AND 23;
```

name	age
Soliso	22
Sino	20
Vivi	23

END OF CLASS

16/06/26.

Sorting statement

↳ It's a way of sorting out data, when sorting we use:

ORDER BY statement

↳ 2 main ways to sort data, we can sort our data either in Ascending order (ASC) or Descending order (DESC)

↳ sorting doesn't remove or add anything new into our data - it just re-arrange the rows in our data

↳ Ascending → it's going up from smallest to biggest / lowest to highest.
order (ASC)

↳ Descending order (DESC) → it's going down from highest to lowest.

Syntax :
SELECT *
FROM table_name
WHERE condition
ORDER BY column_name ASC;
DESC

e.g

students

student_id	name	Course	Age	City
1	Thelbo	Data analytics	22	JHB
2	Sarah	Data Science	19	PTA
3	John	Data analytics	25	ABN
4	Lerato	Data analytics	20	JHB
5	Musa	Data Science	23	JHB

ASC

```
SELECT name,  
       age  
FROM students  
ORDER BY age ASC;
```



name	age
Sarah	19
Lerato	20
Thabo	22
Musa	23
John	25

DESC

```
SELECT name,  
       age  
FROM students  
ORDER BY age DESC;
```



name	age
John	25
Musa	23
Thabo	22
Lerato	20
Sarah	19

Aggregate Functions

↳ used to perform a calculation on rows of a specific column and returns a single value / a single value per group.

Function	Description
COUNT ()	Counts the number of rows/unique identifiers
SUM ()	Calculates the total of a numeric column
AVG ()	Calculates the average of a numeric column
MAX ()	Finds the maximum value
MIN ()	Finds the minimum value

min & max

eg

Employees

Emp_id	Name	Salary
101	John	20
102	Mary	40
103	David	35

```
SELECT MIN (salary) AS min_salary  
FROM employees;
```

min_salary
20

(alias)
pick name for the column

```
SELECT MAX (salary) AS max_salary  
FROM employees;
```

max_salary
40

AS → is a way to rename a column after applying a function
(descriptive name)

Aggregate functions

17/06/26

e.g

Students

Student_id	Name	Subject
101	Rofu	Science
102	Jeff	Commerce
103	Mark	Commerce
104	Elon	Science
105	Steve	History
106	Donald	Commerce

SELECT COUNT (DISTINCT student_id) AS number_of_students
FROM students;

↳ count is number of students in the class using their ID

number_of_students
6

- We COUNT() something that is uniquely identified
e.g id
- We do not count the revenue, we SUM() it
- We do not sum id we COUNT()

- **GROUP BY statement** → only needed when using aggregate functions
↳ is a statement that allows us to group our data into specific categories

use the table
e.g **SELECT** **subject**,
COUNT (DISTINCT student_id) AS number_of_students
FROM students

GROUP BY **subject**;

↳ we group by a column that is not aggregated in the select statement
↳ we commonly group by a categorical column

Subject	number_of_students
Science	2
Commerce	3
History	1

e.g 2

Sales			
Sale_id	Customer_name	category	Revenue
1	John	Electronics	120
2	Mary	Food	80
3	John John	Electronics	150
4	Peter	Clothing	100

Q Write an SQL query to calculate the total revenue for each category

```
SELECT category,  
       SUM(revenue) AS total_revenue  
FROM sales  
GROUP BY category;
```



Category	total_revenue
Electronics	270
Food	80
Clothing	100

NB Group by a column that is not aggregated, ^{use} a column containing categorical data

SQL fundamentals (HAVING Statement)

18/06/26

Recap:

Aggregate functions

→ used to summarize data → extract specific information

↳ perform calculations on multiple rows of a specific column and returns a single value / single value per group

↳ consolidate data

① COUNT () - we count unique identifiers (ID)

② SUM () - we sum numeric data (revenue)

③ AVG () - we average numeric data (revenue)

④ MIN () - looks for min value in a column

⑤ MAX () - looks for max value in a column

GROUP BY statement

~~SELECT~~ / categorical - column / Aggregate function (column) AS descriptive column name

SYNTAX

SELECT categorical - column

→ e.g gender, race, category
→ column in words

→ Aggregate function (column) AS descriptive column name

FROM table - name

→ numerical column

GROUP BY categorical - column;

↳ we group by a column that is not aggregated

HAVING statement

- ↳ It is used to filter data that is aggregated
- ↳ applied to data that is already aggregated
- ↳ It is used to filter grouped results after using a GROUP BY
- ↳ similar to WHERE statement, both used for filtering data

Diff between HAVING & WHERE statement

- **HAVING** - filters data after grouping by GROUP BY statement.
- **WHERE** - filters rows before grouping

e.g.

Sales

cust_id	cust_name	product	amount
101	Rofhi	Laptop	5000
101	Rofhi	Mouse	300
102	Abel	keyboard	700
102	Abel	Monitor	2500
103	Giora	Mouse	300

Q₁ write an SQL query to find customers who spent more than 3000

Q₂ How much ~~did~~ a specific customer (e.g Rofhi) spend.

A₂

```
SELECT cust_name,  
       SUM (amount) AS total_spent  
FROM sales  
GROUP BY cust_name;
```

↓

cust_name	total_spent
Rofhi	5300

Continued

Abel	3200
Ciara	300

A1 SELECT cust_name,
SUM(amount) AS total_spent
FROM sales
GROUP BY cust_name
HAVING SUM(amount) > 3000

↓

cust_name	total_spent
Rofhi	5300
Abel	3200

22/06/20

SQL fundamentals

Aggregate functions Exercise 2

① SELECT COUNT (*) AS total_employees
FROM employees;

↓

total_employees
10

② SELECT SUM(salary) AS total_IT_salary
FROM employees
WHERE department = 'IT';

↓

total_IT_salary
220000

④ SELECT MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary
FROM employees;

↓

highest_salary	lowest_salary
62000	4800

⑤ SELECT department,
SUM(salary) AS total_salary
FROM employees
GROUP BY department;

↓

department	total_salary
IT	
Finance	
marketing	
HR	

⑥ SELECT city,
COUNT (DISTINCT id) AS employee_cnt
FROM employees
GROUP BY city

↓

city	employee_cnt
New York	1
Chicago	3
San Francisco	2
Houston	1
Los Angeles	2

⑦ SELECT department,
SUM (salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM (salary) > 100000;

↓

department	total_salary
IT	22000
Finance	119000
Marketing	105000

LIMIT statement

↳ It is a way of restricting the number of rows you want in your final output

e.g. Employees

id	name
101	Rofu
102	Mbali
103	Zuma

```
SELECT *  
FROM employees  
LIMIT 2;
```

id	name
101	Rofu
102	Mbali

23/06/26

- CASE statements ^{like IF ELSE}

- ↳ allows us to create a new column based on logical conditions applied on other columns
- ↳ allows us to classify our numerical data into categorical groupings.

syntax : `SELECT column1, &`
`CASE`
`WHEN condition1 THEN result1`
`WHEN condition2 THEN result2`
`ELSE default result`
`END AS column_name (new column)`
`FROM table_name ;`

e.g.

employees

emp-id	name	Salary
101	John	3000
102	Peter	7000
103	Samson	1200
104	David	4000
105	Goliath	300

what to classify them
as low, mid & high
salary earnings

business logic { $Salary < 5000 = \text{low}$
 $Salary < 500 \text{ \& } 10000 = \text{med}$
 $Salary > 10000 = \text{high}$

`SELECT name,`
`Salary,`

`CASE`

`WHEN salary < 5000 THEN 'Low'`

`WHEN salary BETWEEN 5000 AND 10000 THEN 'med'`

`WHEN salary > 10000 THEN 'High'`

`END AS salary-bracket`
`FROM employees`

→ The case statement is placed within the select statement
 → case statement creates a new column

output :

name	salary	salary_bracket
John	3000	low
Peter	7000	med
Samson	12000	High
David	4000	low
Goliath	800	low

Q

products			Price_category
product_id	product_name	price	
1	Laptop	1200.00	Expensive
2	Phone	800.00	mid-range
3	keyboard	45.00	budget
4	Monitor	300.00	mid-range
5	Mouse	25.00	budget

Classify each product by price:

- 'Expensive' if price > 1000
- 'mid-range' if price between 100 and 1000
- 'Budget' if price < 100

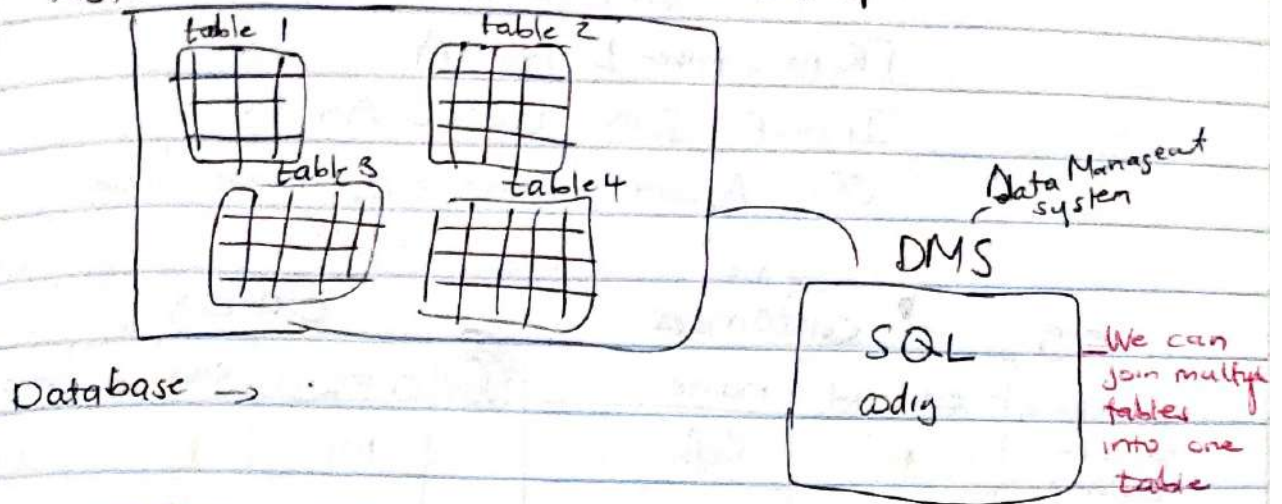
A

```
SELECT product_name,  
       price,  
       CASE WHEN price > 1000 THEN 'Expensive'  
            WHEN price BETWEEN 100 AND 1000 THEN 'mid-range'  
            WHEN price < 100 THEN 'Budget'  
       END AS price_category  
FROM products;
```

SQL JOINS ^{intro}

24/06/21

^{has multiple data bases}
In a data warehouse ^{Relational Database central data store} we have multiple tables. The diff tables have some form of relationship

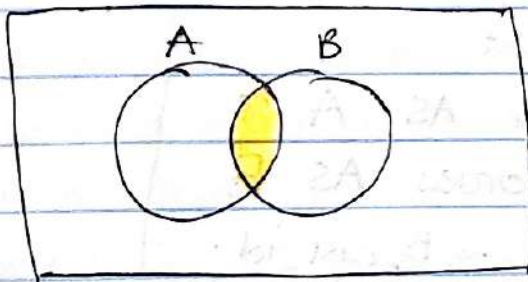


SQL JOINS

↳ is meant to join multiple tables based on common columns (they share something common)

① INNER JOIN

↳ returns rows where there is a match on both the tables



- we join tables on columns but the join affects the output rows
- when we join table we can select any column between the tables we are joining

~~eg: join 2~~ SELECT A.column1,
A.column2
B.column3,
B.column4

syntax : `SELECT A.column 1,
A.column 2,
B.column 3,
B.column 4
FROM table 1 AS A
INNER JOIN table 2 AS B
ON A.common_column = B.common_column;`

e.g

customers		orders	
cust_id	name	order_id	cust_id
1	Rofhi	101	1
2	Mbali	102	4
•		103	5
		104	1
			300

is unique
for each
customer

primary key on this table

primary key

is a foreign key on
this table
is not unique
it can be repeated

SQL code to join 2 table into one

dimension
table
contains
descriptive data
for here doesn't
change frequently
(static)

```
SELECT A.cust_id,  
A.name,  
B.amount  
FROM customers AS A  
INNER JOIN orders AS B  
ON A.cust_id = B.cust_id;
```

fact table
• numerical
data
• used for analysis

cust_id	name	amount
1	Rofhi	500
1	Rofhi	300

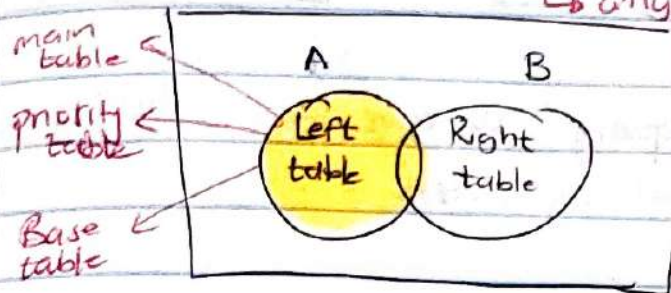
② SQL LEFT JOINS

25/06/26

↳ returns all rows from the left table (A) even if there is no matching record on the right table (B).

↳ In our final outcome, we need all rows in the left table and only return only matching results from (B)

↳ only take the things you want from B



Syntax :

```
SELECT column 1,  
        Column 2,  
        Column 3  
FROM table1 AS A  
LEFT JOIN table2 AS B  
ON A.common_column = B.common_column;
```

Annotations: 'left table' points to 'table1 AS A'; 'right table' points to 'table2 AS B'.

↳ A LEFT JOIN returns all ^{rows} records from the left table, and only the matching ^{rows} records from the right table

↳ When there is no match in the right table we still keep the left table row but the right table columns will show NULL.

eg customers_table

cust_id	cust_name
1	Rofhiwa
2	Lerato
3	Mbali

orders_table

order_id	cust_id	amount
101	1	500
102	2	300

Q Write an SQL query to give me a list of all customers and what they ordered

A

```

SELECT cust_id,
       cust_name,
       amount,
       order_id
FROM customers_table AS A
LEFT JOIN orders_table AS B
ON A.cust_id = B.cust_id
  
```

left table

all the rows on our left table will be returned.

cust_id	cust_name	amount	order_id
1	Rofhiwa	500	101
2	Lerato	300	102
3	Mbali	NULL	NULL

NULL → empty space
 → missing value

13/07/2026

8-9pm Consultation on Bright TV Analysis

* Mind
Ready to project requirements

NULL functions

→ NULL doesn't mean zero

- ↳ A NULL is an empty space / missing value
- ↳ A NULL is a blank
- ↳ NULL functions help us to deal with NULL values
- ↳ NULL simply means there is no value (the data is missing / unknown)

sales

product	Amount
Apple	100
Orange	NULL
Banana	20

→ $100 + \text{null} + 20$

Q find the total revenue

```
SELECT SUM (Amount) AS revenue  
FROM sales
```

Now we want to deal with the null value :

1 IF NULL (expression, replacement_value)

→ column_name

↳ is a way to replace all NULL values with a value from table ;

Syntax :
SELECT
IF NULL (column_name, value)
FROM table ;

A

```
SELECT product,  
IF NULL (Amount, 0) AS Amount  
FROM sales
```

↓

product	Amount
Apple	100
Orange	0
Banana	20

* **IFNULL** is a way to replace a null value with a value

②. **COALESCE** (col1, col2, col3, col4, ..., colz)

- ↳ is a way to replace null values by a value from alternative column or a specified value
- ↳ we commonly use it in our SELECT statement
- ↳ simply means if the value is NULL, use something else instead

eg customers

cust_id	cust_name	phone_number	whatsapp-number
101	John	071 111 1111	null
102	Mary	null	082 222 2222
103	Peter	null	null
104	Aberigo	0712345678	0825432101

Q Write an SQL query to give me the name of the customer and their contact details

A

Syntax: COALESCE (column, replacement)

SELECT cust_name,
COALESCE

SELECT cust_name,
COALESCE (phone_number, whatsapp-number, 0000)
AS contact_number,
FROM customers;

NB checking from left to right

∴ phone_number column is priority, if there's no number then write whatsapp number

cust_name	contact_number
John	071 111 1111
Mary	082 222 2222
Peter	0000
Aberigo	071 234 5678

③ IS NULL → not a function

↳ is a way to check if there is a null value in a specific column

↳ we can use it in our WHERE statements

c.g customers table from previous pg

Q Write a query to show customers who's phonenumber is null

```
SELECT name cust_name,  
       phone_number  
FROM   customers  
WHERE  phone_number IS NULL;
```

↓

cust_name	phone_number
Mary	null
Peter	null

```
Syntax : SELECT column_name  
         FROM table_name  
         WHERE column_name IS NULL;
```

④

IS NOT NULL

↳ It checks if a value is **not** NULL

↳ it like it's asking "Does this value exist?"

```
syntax: SELECT column_name  
        FROM table_name  
        WHERE column_name IS NOT NULL;
```

↳ returns columns with no NULL values

* ~~DATE~~ DATE Functions

↳ Help us manipulate the date column

MONTH

SELECT MONTH ('2026-07-05') AS month_dt; ⇒ 07

① TO_DATE

↳ ensure that a date is read as a date

↳ converts a string into a date

in viewership table it extracted the date ('MM-MM-YY') from a Timestamp formatted date column

↳ syntax : TO_DATE (column_name) AS new_column_name

e.g. TO_DATE (RecvdDate2) AS watch_date

RecvdDate2	watch_date
2016-01-01T15:01	2016-01-01
2026-07-14T19:01	2026-07-14

② DAYNAME

↳ extracts what day of the week (mon-sun)

↳ syntax: DAYNAME (column_name) AS Day_name

e.g. DAYNAME (TO_DATE (RecvdDate2)) AS Day_name

watch_date	Day_name
2026-07-14	Tue

↳ This tells us what day of the week was 14/07/26

e.g. SELECT DAYNAME ('2026-07-05') AS day_name;

③ MONTHNAME

↳ SELECT MONTH ('2026-07-05') AS month_dt; ⇒ 07

↳ extracts the name of the month

↳ syntax : MONTHNAME (column_name) AS month_name

e.g. MONTHNAME (watch_date) AS month_name

watch_date	month_name
2026-07-14	Jul

④ YEAR

↳ extracts the year from the date

syntax: YEAR (column_name) AS event_year

e.g. YEAR (watch_date) AS event_year

↓

watch_date	Event_year
2026-07-14	2026

e.g. SELECT YEAR ('2026-07-05') AS year_dt;

↓
2026

⑤ DAY

↳ Extracts the day from the date

syntax: DAY (column_name) AS event_day

e.g. DAY (watch_date) AS event_day

↓

Watch-date	event_ dt
2026-07-14	14

e.g. SELECT DAY ('2026-07-05') AS day-dt; → 05

⑥ ~~CURRENT~~ CURRENT_DATE()

↳ it gives us today's date

e.g. SELECT CURRENT_DATE() AS today-dt;

⑦ CURRENT_TIMESTAMP()

↳ gives me the current date and time

SELECT CURRENT_TIMESTAMP() AS dt_time;

⑧ DATEDIFF (end_date, start_date)

↳ used to calculate diff between 2 dates (end_date - start_date = value)

SELECT DATEDIFF ('2026-07-05', '2026-07-01') AS dt_difference;

↓

dt-difference
4

CHAR = character

20/07/2026

9) TO_CHAR()

↳ converts date into a string

SELECT TO_CHAR(TO_DATE('2026-07-17'), 'yyyymmdd') AS Month_d

↓

202607

yyyymmdd

↳ the system

recognises the date in this format

10) DATE_ADD (date, number_of_days)

↳ Measure = DAY, MONTH, YEAR

SELECT DATE_ADD('2026-07-05', 5) AS add_5_days;

11) DATE_SUB (date, number_of_days)

SELECT DATE_SUB('2026-07-05', 2) AS removing_2_days;

12) DAYOFWEEK

13) ADD_MONTHS

↳ add months to the date

SELECT ADD_MONTH('2026-07-05', 2) AS adding_2_months

ADD_MONTHS (date, number_of_months)

Date Functions

~~2026-05-01~~

2026-01-05

Format	Meaning	Example
M	Month number	1
MM	Month number, 2 digits	01
MMM	Short month name	Jan
Month MMMM	Full month name	January
yy / YY	2-digit year	26
yyyy / YYYY	4 digit year	2026

TO_CHAR (column_name 'MMMM yyyy') AS col_month_year

Q1

col_month_year
January 2026

OR

TO_CHAR (column_name 'Month YYYY') AS col_month_year

↓

col_month_year
January 2026

Q2

customer	DOB	customer_age
Lisa	1998-05-10	? 28

SELECT customer,
DOB

FLOOR (DATEDIFF(CURRENT_DATE(), DOB) / 365.25) AS customer_age

FROM table_name;

Explanation:

* CURRENT_DATE() → today's date (2026-08-09)

* DOB → date of birth (1998-05-10)

* { DATEDIFF() → How many days between 2 dates
DATEDIFF(CURRENT_DATE(), DOB) → 10,300 days

* / 365.25 → converts the difference from days to years

→ DATEDIFF(CURRENT_DATE(), DOB) / 365.25 → 28.2 years

* FLOOR → rounds DOWN the number to nearest whole number ⇒ 28

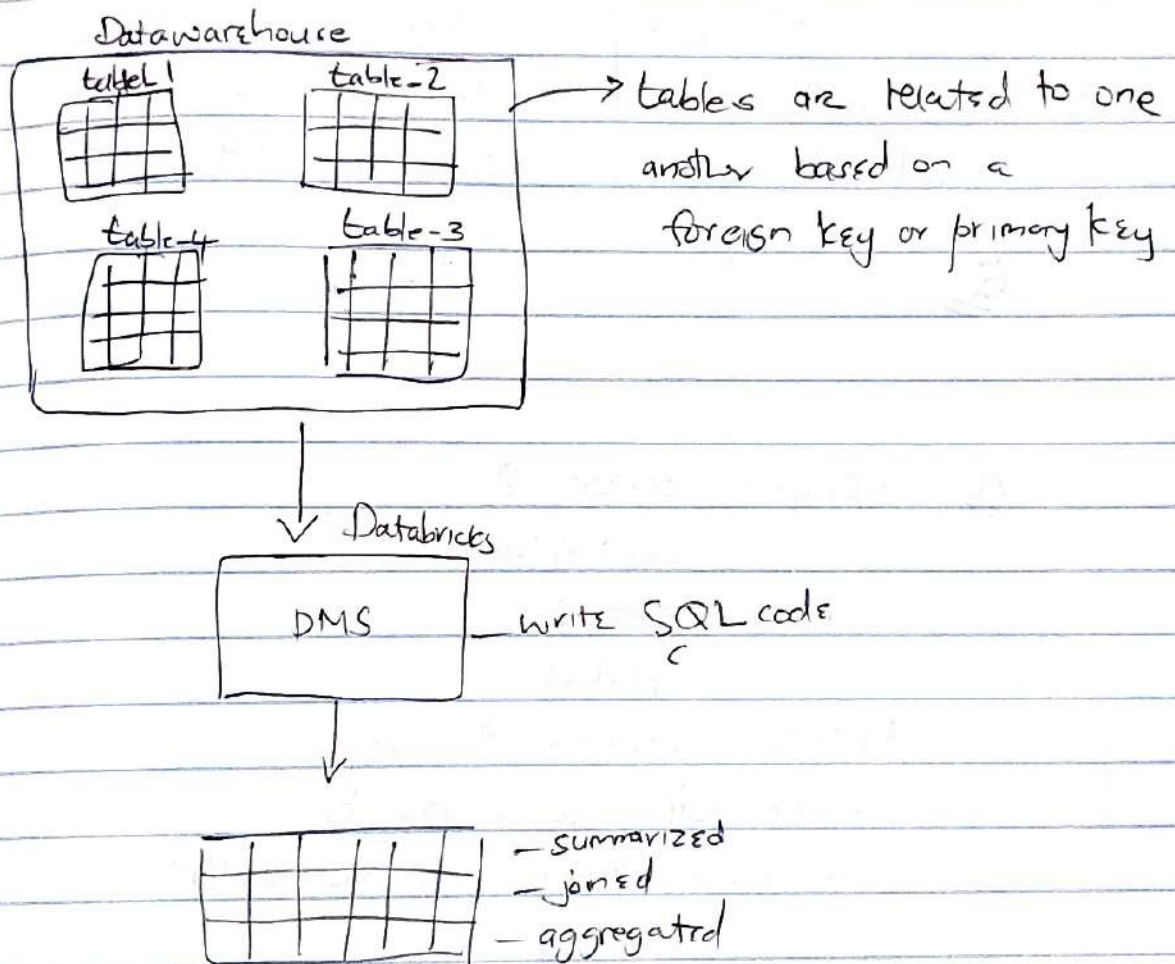
JOINS

14/07/2026

↳ We want to combine multiple tables into one single table

↳ We join tables based on a common column between the diff tables

* In a data warehouse you will have multiple tables that are related to one another.



How we join tables?

① INNER JOIN

↳ joins tables ~~into one~~ ^{returns} only rows that have matching values on both tables

Syntax :

```
SELECT col1, col2, col3  
FROM table1 AS A  
INNER JOIN table2 AS B  
ON A.common_column = B.common_column;
```

e.g

Customers		orders		
customer ID	customer Name	order ID	customer ID	Product
1	Thabo	101	1	Laptop
2	Sarah	102	2	phone
3	Lerato	103	2	Headphones
4	John	104	5	Tablet

Q

```
A SELECT customerID,  
customerName,  
orderID,  
product  
FROM customers AS A  
INNER JOIN orders AS B  
ON A.customerID = B.customerID;
```

↓

customerID	customerName	orderID	product
1	Thabo	101	Laptop
2	Sarah	102	Phone
2	Sarah	103	Headphones

You can join on other columns also by using the AND (multiple)

e.g $A.customerID = B.customerID$ AND $A.dateID = B.dateID$

* Download the SQL notes picture under 'Additional resources'

15/07/2026

③ FULL OUTER JOIN

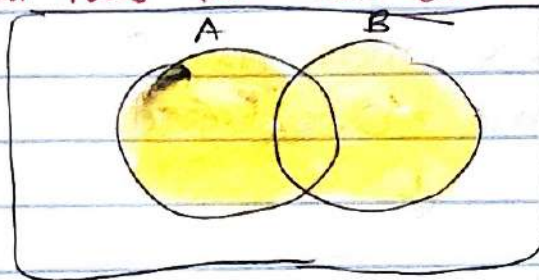
↳ returns all ~~rows~~ matching rows

↳ All the rows that are matching on the common columns are combined into a single row

↳ returns all unmatched rows

↳ where there's no match we will put a NULL

returns all rows from both tables



syntax :

```
SELECT col1, col2, col3
```

```
FROM table1 AS A
```

```
FULL OUTER JOIN table2 AS B
```

```
ON A.common_column = B.common_column;
```

e.g

customers

customerID	customerName
1	Thabo
2	Sarah
3	Lerato
4	John

orders

orderID	customerID	Product
101	1	Laptop
102	2	Phone
103	2	Headphones
104	5	Tablet

```
SELECT customerID,  
       customerName,  
       orderID,  
       product
```

```
FROM customers AS C  
FULL OUTER JOIN orders AS O  
ON C.customerID = O.customerID;
```



customerID	customerName	orderID	Product
1	Thabo	101	Laptop
2	Sarah	102	Phone
2	Sarah	103	Headphones
3	Lerato	NULL	NULL
4	John	NULL	NULL
5	NULL	104	Tablet

removes duplicates

doesn't remove duplicates

UNION

and

UNION ALL

↳ combines the results of two **SELECT** statements by stacking the rows

- The **SELECT** statements must have the same number of columns
- The columns must be in the same order
- The columns must have the same data types

current_students

student_id	StudentName
1	Thabo
2	Sarah
3	Lerato

new_students

Student_id	StudentName
3	Lerato
4	John
5	Mary

NB The order of tables must be the same on both **SELECT**s

UNION

```
SELECT student_id,  
       StudentName  
FROM   current_students  
UNION  
SELECT student_id,  
       StudentName  
FROM   new_students;
```

student_id	StudentName
1	Thabo
2	Sarah
3	Lerato
4	John
5	Mary

NB Lerato will be written once the other doesn't appear since it's a duplicate

UNION ALL

```
SELECT student_id,  
       StudentName  
FROM   current_students  
UNION ALL  
SELECT student_id,  
       StudentName  
FROM   new_students;
```

Student_id	Student Name
1	Thabo
2	Sarah
3	Lerato
4	John
5	Mary
3	Lerato

Lerato appears twice since duplicates not removed

Filtering Data

• WILD CARDS

↳ If there are similarities existing in a column

↳ we want to filter data based on a specific column pattern in a column

* underscore ^{1 or more} represents a single letter ^{value} (it could be a letter ⁽¹⁻²⁾ or a number (0-9))

* % ^{10 or more} (percent sign) → represents 0 or more characters

• LIKE operator

↳ allows us to filter our data by specifying what something is like on our column

Syntax:

SELECT col1, col2, col3

FROM table_name

WHERE column LIKE 'pattern'

e.g

customers

cust_id	name	product	amount
101	John ✓	Apple	20
102	Sarah	Banana	120
103	Michael	Orange	200
104	Johnny ✓	Banana	100
105	Jay Jay	Kiwi	50

NS

look for
someone who's
name start with
Jo

SELECT name,
amount

FROM customers

WHERE name LIKE 'Jo%';

name	amount
John	20
Johnny	100

25/08/20

Window Functions

-> **PARTITION** -> is to breakdown something big into small pieces so you are able to work/do something with it.

Window Functions

26/08/26

↳ A window function lets you compute a value for each row while "looking at" other rows that are related

↳ creates a new column when used in the ~~select~~ SELECT statement
↳ we use it with the OVER() statement.

• Partitions

↳ is to breakdown things into groups of similarity

↳ groups data that is similar to one another without collapsing the rows

Syntax: window_function (expression)

OVER (PARTITION BY <grouping columns>

ORDER BY <sorting columns>) AS

descriptive_name

* PARTITION BY → breakdown each group

* ORDER BY → define the flow/sequence

LEAD function

- finds the NEXT row
- looks Ahead
- future

LAG function

- finds the PREVIOUS row
- looks behind
- past

eg Sales

sale_id	salesperson	sale_date	Revenue
1	mbali	2026-08-01	1000
2	mbali	2026-08-02	1200
3	mbali	2026-08-03	900
4	John	2026-08-04	800
5	John	2026-08-05	950

How LEAD function work

```
SELECT sales_person,  
       sales_date,  
       revenue,  
       LEAD (revenue)  
         OVER (PARTITION BY sales_person  
              ORDER BY sale_date ASC  
              ) AS nextday_revenue  
FROM sales;
```

↓

sales_person	sales_date	revenue	nextday_revenue
mbali	2026-08-01	1000	1200
mbali	2026-08-02	1200	900
mbali	2026-08-03	900	null
John	2026-08-04	800	950
John	2026-08-05	950	null

LAG

use same table

sales

SELECT salesperson,
sales_date,
revenue,

LAG (revenue)

OVER (PARTITION BY salesperson
ORDER BY sales_date ASC
) AS prevday_revenue

FROM sales;



salesperson	sales_date	revenue	prevday_revenue
mbali	2026-08-01	1000	null
mbali	2026-08-02	1200	1000
mbali	2026-08-03	900	1200
John	2026-08-04	800	null
John	2026-08-05	950	800